

PROJECT REPORT

Blog Website

A Full-Stack Web Application built with Node.js and Express

Repository:

<https://github.com/koyyadakrithika-spec/website>

Prepared for: Full Stack Development Coursework / Portfolio Documentation

Author of Repository: koyyadakrithika-spec

Report Prepared: August 2026

Technology Stack: HTML5 • CSS3 • JavaScript • Node.js • Express.js

1. Introduction

1.1 Overview

This report documents the design, architecture, and implementation of the “Blog Website” project, a full-stack web application hosted in the GitHub repository `archanadeema08/full-stack`. The project demonstrates a complete client-server workflow in which a Node.js and Express backend exposes a REST-style API for managing blog posts, while a lightweight HTML, CSS, and JavaScript frontend consumes that API to let a user view and create blog entries through the browser.

The application follows a classic three-tier structure: a presentation layer built with static HTML/CSS/JavaScript served from the public directory, an application layer implemented in `server.js` using the Express framework, and a data layer that, in the current version of the project, is an in-memory JavaScript array rather than a persistent database. This makes the project an excellent teaching example of how HTTP requests, middleware, and DOM-driven client logic fit together, while also highlighting the next steps required to turn it into a production-grade system.

1.2 Purpose of this Report

The purpose of this report is to provide a structured, in-depth walkthrough of the repository suitable for academic submission, code review, or portfolio documentation. It covers the problem the project solves, the requirements it satisfies, the system design and architecture, the technology stack, the implementation details of every file in the repository, the testing approach, identified limitations, and recommendations for future enhancement.

1.3 Scope of the Project

The scope of the Blog Website project, as implemented in the repository, includes:

- Creating new blog posts with a title, author, and content through a web form.
- Viewing all submitted blog posts on the home page.
- Editing an existing blog post through a dedicated API endpoint.
- Deleting a blog post through a dedicated API endpoint.
- Serving static frontend assets (HTML, CSS, JavaScript) directly from the Express server.

The scope explicitly excludes user authentication, persistent database storage, and deployment automation in the current version, since none of these are present in the repository at the time of writing. These are discussed later as recommended future enhancements.

2. Project Objectives

The Blog Website project was undertaken with the following objectives in mind:

1. To design and implement a functional REST API using Node.js and the Express.js framework that supports the four fundamental CRUD operations — Create, Read, Update, and Delete — for a blog resource.
2. To build a simple, dependency-free frontend using vanilla HTML, CSS, and JavaScript that communicates with the backend using the Fetch API.
3. To demonstrate correct use of Express middleware, including JSON body parsing, URL-encoded form parsing, and static file serving.
4. To structure the codebase in a way that separates backend logic (server.js) from frontend assets (the public folder), reflecting standard full-stack project conventions.
5. To provide a foundation that can later be extended with a persistent database, authentication, and deployment, making the project a stepping stone toward a production-ready blogging platform.
6. To serve as a learning and demonstration project for full-stack JavaScript development, suitable for a student portfolio or coursework submission.

3. Background and Existing System Review

3.1 The Problem Being Addressed

Blogging platforms allow individuals to publish written content to an audience. Full-featured platforms such as WordPress, Medium, and Ghost provide rich functionality including user accounts, comments, media management, and analytics, but they are also large, complex systems with significant learning curves. For a student or a developer wanting to understand how a blogging system works under the hood, building a minimal version from first principles is a valuable exercise.

The Blog Website project addresses this by stripping the concept of a blogging platform down to its essential operations: creating a post and viewing the list of posts, with editing and deletion supported at the API level. This minimal feature set makes the underlying client-server communication pattern easy to trace from the browser form, through the JavaScript fetch call, into the Express route handler, and back again as a JSON response.

3.2 Comparable Approaches

Conceptually, the project is comparable to introductory full-stack tutorials that pair Express with a simple frontend before introducing a database layer such as MongoDB or PostgreSQL. Many bootcamp curricula follow this exact progression: first build the API with in-memory storage to understand routing and middleware, then swap the storage layer for a real database once the request/response cycle is well understood. The current repository sits at the first of these two stages.

3.3 Why Express.js

Express.js was chosen as the backend framework because it is the most widely adopted minimalist web framework for Node.js. It provides simple, unopinionated routing, a large middleware ecosystem, and a gentle learning curve, making it well suited to a small CRUD-style project such as this one. The package.json file confirms that express (version ^4.22.2) is the only runtime dependency declared, keeping the project lightweight and easy to install.

4. System Analysis and Requirements

4.1 Functional Requirements

Based on the routes implemented in `server.js`, the system satisfies the following functional requirements:

ID	Requirement	Implemented By
FR-1	The system shall allow a user to view all existing blog posts.	GET /blogs
FR-2	The system shall allow a user to submit a new blog post with a title, author, and content.	POST /add-blog
FR-3	The system shall allow an existing blog post to be updated by its index/ID.	PUT /edit-blog/:id
FR-4	The system shall allow an existing blog post to be removed by its index/ID.	DELETE /delete-blog/:id
FR-5	The system shall serve static frontend files (HTML, CSS, JS) to the browser.	<code>express.static("public")</code>
FR-6	The system shall validate that a requested ID exists before editing or deleting.	<code>isNaN</code> / bounds check in PUT & DELETE handlers

4.2 Non-Functional Requirements

- Usability: The interface should be simple enough that a first-time visitor can add a blog post without instructions.
- Performance: Because data is held in memory, read and write operations complete in constant or near-constant time for small data sets.
- Maintainability: Code should be organized so that backend and frontend concerns are clearly separated, which the repository structure already achieves.
- Portability: The application should run on any machine with Node.js installed, without external service dependencies.
- Extensibility: The architecture should allow a database layer, authentication, and additional routes to be added without a full rewrite.

4.3 Hardware and Software Requirements

Software Requirements

- Node.js (v14 or later recommended) and npm.
- Express.js ^4.22.2 (installed automatically via `npm install`).
- A modern web browser (Chrome, Firefox, Edge, or Safari) with JavaScript enabled.
- A code editor such as Visual Studio Code for development.

Hardware Requirements

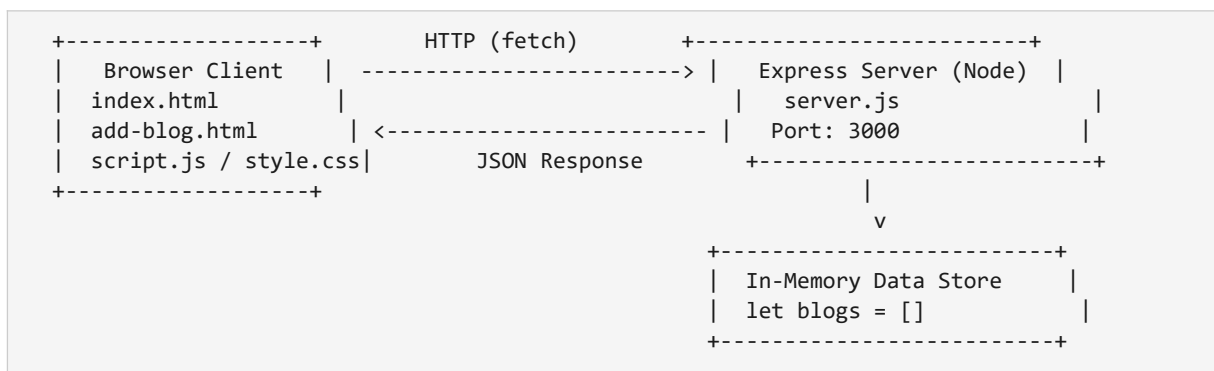
- Any standard development machine capable of running Node.js: minimum 4 GB RAM and a dual-core processor.
- No specialized hardware, GPU, or database server is required in the current version.

5. System Design and Architecture

5.1 Architectural Style

The application follows a client-server architecture with a clear separation between the presentation layer and the application/data layer. The Express server plays a dual role: it serves static frontend assets and it exposes a JSON REST API consumed by that same frontend using JavaScript's Fetch API. This is a common pattern for small full-stack JavaScript applications before a dedicated single-page-application framework or a separate frontend deployment is introduced.

5.2 High-Level Architecture Diagram (Textual Representation)



5.3 Module Description

Module / File	Responsibility
server.js	Application entry point. Configures Express, middleware, and all API routes; manages the in-memory blogs array; starts the HTTP server on port 3000.
public/index.html	Home page. Displays the page header, navigation bar, and a container (#blogList) intended to render the list of blog posts.
public/add-blog.html	Form page that collects a blog title, author, and content from the user and submits it to the backend.
public/script.js	Client-side logic. Listens for the add-blog form submission, prevents the default page reload, sends a POST request to /add-blog using fetch, and alerts the user with the server's response message.
public/style.css	Presentation layer styling for the header, navigation, form, and container elements shared across both HTML pages.
package.json	Project manifest. Declares the project name (blog-website), the entry point (server.js), the start script, and the express dependency.

5.4 Data Model

The project does not use a formal database schema; instead, each blog post is represented as a plain JavaScript object held in an array. The implicit data model, inferred directly from the request bodies handled in `server.js`, is as follows:

Field	Type	Description
title	String	The headline of the blog post, supplied by the user through the form.
author	String	The name of the person who wrote the post.
content	String	The main body text of the blog post.
id (implicit)	Number (array index)	Not stored explicitly; the position of the post within the <code>blogs</code> array is used as its identifier for edit and delete operations.

Because the identifier is simply the array index, this data model is fragile: deleting a post shifts the indices of every post after it, which can cause a client holding a stale index to edit or delete the wrong post. This is discussed further in the Limitations section.

6. Technology Stack

6.1 Backend

- Node.js — JavaScript runtime environment used to execute server.js outside the browser.
- Express.js (^4.22.2) — minimalist web framework used for routing and middleware.
- Built-in Express middleware: `express.json()` for parsing JSON request bodies and `express.urlencoded({ extended: true })` for parsing form-encoded bodies.
- `express.static("public")` — middleware used to serve the frontend files directly from the server.

6.2 Frontend

- HTML5 — structures the Home page (`index.html`) and the Add Blog page (`add-blog.html`).
- CSS3 — provides layout and visual styling via `style.css`.
- Vanilla JavaScript (ES6+) — `script.js` uses the Fetch API and `async/await` to communicate with the backend without any frontend framework or build tool.

6.3 Tooling and Package Management

- npm — used to install and manage the express dependency, as declared in `package.json` and `package-lock.json`.
- Git and GitHub — used for version control and hosting the source repository.

6.4 Why This Stack Was Chosen

This stack was selected for its minimalism and its low barrier to entry. Node.js and Express together allow a JavaScript developer to write both the client and the server in the same language, eliminating context-switching between languages. Because the frontend uses no framework or bundler, the project can be run and understood without any build step, which makes it particularly suitable for learning purposes and for quickly demonstrating full-stack concepts.

7. Implementation Details

7.1 Project Structure

```
full-stack/
├── node_modules/      (installed dependencies)
├── public/
│   ├── index.html     (Home page – lists blogs)
│   ├── add-blog.html  (Form to create a new blog)
│   ├── script.js      (Client-side fetch logic)
│   └── style.css       (Shared styling)
├── package.json       (Project manifest)
├── package-lock.json  (Locked dependency tree)
└── server.js          (Express application & API routes)
```

7.2 Server Bootstrap and Middleware

The server begins by importing Express, creating an application instance, and defining the port. Three middleware functions are registered before any route: a JSON body parser, a URL-encoded body parser, and the static file server pointed at the public directory. This ordering is important — Express evaluates middleware and routes in the order they are declared, so the parsers must be registered before any route handler that reads `req.body`.

```
const express = require("express");

const app = express();
const PORT = 3000;

// Middleware
app.use(express.json());
app.use(express.urlencoded({ extended: true }));
app.use(express.static("public"));

// Store blogs in a JavaScript array
let blogs = [];
```

7.3 API Endpoint Reference

Method	Route	Description	Response
GET	/blogs	Returns the full array of blog posts currently stored in memory.	200 OK — JSON array of blog objects
POST	/add-blog	Creates a new blog post from title, author, and content in the request body and appends it to the array.	201 Created — success message + updated array

Method	Route	Description	Response
PUT	/edit-blog/:id	Replaces the blog post at the given array index with new field values.	200 OK, or 404 if the id is invalid
DELETE	/delete-blog/:id	Removes the blog post at the given array index using Array.splice.	200 OK, or 404 if the id is invalid

7.4 Create Operation — POST /add-blog

The add-blog endpoint reads title, author, and content from the JSON request body, constructs a new blog object, and pushes it into the in-memory blogs array. It then responds with HTTP status 201 (Created) along with a confirmation message and the full, updated list of blogs.

```
app.post("/add-blog", (req, res) => {
  const newBlog = {
    title: req.body.title,
    author: req.body.author,
    content: req.body.content
  };
  blogs.push(newBlog);
  res.status(201).json({
    message: "Blog Added Successfully!",
    blogs: blogs
  });
});
```

7.5 Update Operation — PUT /edit-blog/:id

The edit-blog endpoint parses the :id route parameter as an integer and validates that it is a real number within the bounds of the blogs array. If validation fails, the server responds with a 404 status and a “Blog not found” message. Otherwise, the blog object at that index is replaced entirely with the new values supplied in the request body.

```
app.put("/edit-blog/:id", (req, res) => {
  const id = parseInt(req.params.id);
  if (isNaN(id) || id < 0 || id >= blogs.length) {
    return res.status(404).json({ message: "Blog not found" });
  }
  blogs[id] = {
    title: req.body.title,
    author: req.body.author,
    content: req.body.content
  };
  res.json({ message: "Blog Updated Successfully!", blogs: blogs });
});
```

7.6 Delete Operation — DELETE /delete-blog/:id

The delete-blog endpoint follows the same validation pattern as the edit endpoint, then uses `Array.prototype.splice` to remove exactly one element at the given index, permanently deleting that blog post from memory.

```
app.delete("/delete-blog/:id", (req, res) => {
  const id = parseInt(req.params.id);
  if (isNaN(id) || id < 0 || id >= blogs.length) {
    return res.status(404).json({ message: "Blog not found" });
  }
  blogs.splice(id, 1);
  res.json({ message: "Blog Deleted Successfully!", blogs: blogs });
});
```

7.7 Frontend Implementation

`index.html` defines the Home page: a header, a two-item navigation bar (Home and Add Blog), and an empty container element with the id `blogList`, which is designed to be populated with blog posts. `add-blog.html` defines a form with three inputs — Blog Title, Author, and Content — plus a submit button, all wired to the shared `style.css` for consistent appearance across both pages.

`script.js` contains the client-side logic. It selects the `blogForm` element, and if present, listens for its submit event. On submission it prevents the default browser page reload, reads the three field values from the DOM, and sends them as a JSON payload to the `/add-blog` endpoint using the Fetch API with `async/await`. The JSON response is parsed and its message field is shown to the user via a browser alert, after which the form is reset.

```
const form = document.getElementById("blogForm");

if (form) {
  form.addEventListener("submit", async function (event) {
    event.preventDefault();
    const title = document.getElementById("title").value;
    const author = document.getElementById("author").value;
    const content = document.getElementById("content").value;

    const response = await fetch("/add-blog", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ title, author, content })
    });

    const data = await response.json();
    alert(data.message);
    form.reset();
  });
}
```

7.8 Running the Application

The application is started using the npm start script defined in package.json, which runs node server.js. Once running, the console prints a confirmation message, and the application becomes available in the browser at the address below.

```
$ npm install
$ npm start

> blog-website@1.0.0 start
> node server.js

Server is running on http://localhost:3000
```

8. User Interface Walkthrough

8.1 Home Page (index.html)

The Home page presents the site title “My Blog Website” in the header, followed by a navigation bar linking to the Home page and the Add Blog page. Below the navigation, a heading labeled “All Blogs” introduces the `blogList` container where posts are intended to be displayed. A footer with a copyright notice appears at the bottom of the page, consistent with the Add Blog page.

8.2 Add Blog Page (add-blog.html)

The Add Blog page presents a single form with three fields — Blog Title, Author, and Content (a multi-line textarea) — each marked as required so the browser enforces basic client-side validation before submission. An “Add Blog” button submits the form, which triggers the fetch-based logic in `script.js` to send the data to the backend without a full page reload.

8.3 Visual Design

Both pages share a single stylesheet, `style.css`, giving the application a consistent header, navigation bar, form, and footer treatment across pages. This shared-stylesheet approach keeps the presentation layer DRY (Don't Repeat Yourself) even though the project does not use a templating engine.

8.4 User Flow Summary

7. The user opens the Home page and is shown the site header, navigation, and (intended) list of blogs.
8. The user clicks “Add Blog” in the navigation bar and is taken to the Add Blog form.
9. The user fills in the title, author, and content fields, all of which are required.
10. On submission, `script.js` intercepts the form, sends a POST request to `/add-blog`, and displays the server's confirmation message via an alert dialog.
11. The form resets, ready for another entry, while the new post now exists in the server's in-memory `blogs` array.

9. Testing

9.1 Testing Approach

No automated test suite (such as Jest, Mocha, or Supertest) is present in the repository at the time of this report. Testing of the API can therefore be carried out manually using a REST client such as Postman, Insomnia, or curl, alongside manual exploratory testing of the two HTML pages in a browser. The table below documents a representative manual test plan covering each endpoint.

9.2 Manual Test Cases

Test ID	Scenario	Steps	Expected Result
TC-01	Retrieve empty blog list	Send GET /blogs immediately after server start.	200 OK with an empty JSON array.
TC-02	Add a valid blog post	Send POST /add-blog with title, author, content in the body.	201 Created with a success message and the post included in the returned array.
TC-03	Retrieve populated blog list	Send GET /blogs after adding one or more posts.	200 OK with an array containing all added posts.
TC-04	Edit an existing post	Send PUT /edit-blog/0 with new field values, assuming at least one post exists.	200 OK with the post at index 0 updated.
TC-05	Edit a non-existent post	Send PUT /edit-blog/99 where index 99 does not exist.	404 Not Found with a “Blog not found” message.
TC-06	Delete an existing post	Send DELETE /delete-blog/0, assuming at least one post exists.	200 OK with the post removed from the returned array.
TC-07	Delete a non-existent post	Send DELETE /delete-blog/-1 or an out-of-range index.	404 Not Found with a “Blog not found” message.
TC-08	Submit the Add Blog form in the browser	Fill in all three fields on add-blog.html and click Add Blog.	An alert shows “Blog Added Successfully!” and the form clears.

9.3 Observations from Testing

Manual verification of the code confirms that the four CRUD routes behave as documented and that the validation logic in the PUT and DELETE handlers correctly rejects out-of-range or non-numeric identifiers with a 404 response. However, because index.html's blogList container is not currently wired up to fetch and render data from GET /blogs, a user who adds a post will not see it appear on the Home page without a manual page reload or without additional client-side JavaScript being written to populate that container.

10. Limitations of the Current Implementation

- No persistent storage: all blog posts are stored in a plain JavaScript array in server memory, so every post is permanently lost whenever the Node.js process restarts.
- No database integration: there is no connection to MongoDB, PostgreSQL, MySQL, or any other database engine, which limits the application to a single running instance with no data durability.
- Index-based identifiers: using the array index as an implicit ID means deleting a post shifts the indices of all subsequent posts, which can lead to a client editing or deleting the wrong post if it is relying on a stale index.
- Home page does not render blog data: index.html contains a #blogList container, but no client-side code currently issues a GET /blogs request to populate it, so the list of blogs is not visibly rendered to the user.
- No authentication or authorization: any client can create, edit, or delete any blog post, since there is no login system, session management, or per-user ownership of posts.
- No input sanitization: blog content is stored and would be rendered as-is, which could expose the application to Cross-Site Scripting (XSS) if and when the content is rendered into the DOM.
- No automated tests: the absence of a test suite means regressions could be introduced without being caught automatically during development.
- Single-server, single-process design: the app has no clustering, load balancing, or environment-based configuration (e.g., for production vs. development), and the port is hard-coded rather than read from an environment variable.

11. Recommended Future Enhancements

11.1 Data Persistence

Replace the in-memory blogs array with a real database such as MongoDB (using Mongoose) or a relational database such as PostgreSQL or MySQL (using an ORM like Sequelize or Prisma). This would ensure blog data survives server restarts and can scale beyond a single process.

11.2 Complete the Read Operation on the Frontend

Add a fetch call in script.js (or a new script included on index.html) that requests GET /blogs on page load and dynamically renders each post's title, author, and content inside the #blogList container, completing the Create-Read loop that is already partially built.

11.3 Stable, Unique Identifiers

Assign each blog post a unique ID (for example, a UUID or an auto-incrementing database primary key) instead of relying on its array position, removing the risk of index drift after deletions.

11.4 Authentication and Authorization

Introduce a login system (for example using JSON Web Tokens or session cookies) so that only authenticated authors can create, edit, or delete their own posts, and so that anonymous visitors are limited to read-only access.

11.5 Input Validation and Sanitization

Add server-side validation (for example with a library such as Joi or express-validator) to reject empty or malformed submissions, and sanitize user-supplied HTML content to prevent Cross-Site Scripting attacks.

11.6 Edit and Delete Controls in the UI

Extend the frontend with Edit and Delete buttons next to each rendered blog post, wired to the existing PUT and DELETE endpoints, so that the full CRUD cycle is accessible directly from the browser rather than only through direct API calls.

11.7 Automated Testing

Introduce a testing framework such as Jest combined with Supertest to automatically verify each API route, and add continuous integration (for example with GitHub Actions) so tests run automatically on every push.

11.8 Deployment

Configure the application for deployment to a cloud platform such as Render, Railway, or Heroku, using an environment variable for the port (`process.env.PORT || 3000`) instead of a hard-coded value, so the app can run correctly in a hosted environment.

11.9 UI/UX Improvements

Enhance the visual design with a modern CSS framework or custom design system, add pagination or search for large numbers of posts, and include timestamps for when each post was created or last updated.

12. Conclusion

The Blog Website project in the `archanadeema08/full-stack` repository is a compact but complete demonstration of full-stack JavaScript development. It successfully implements a RESTful backend using Express.js with all four CRUD operations, and it pairs that backend with a simple, dependency-free HTML/CSS/JavaScript frontend that communicates with the API through the Fetch API. The project cleanly separates concerns between the server (`server.js`) and the client assets (the `public` directory), and its `package.json` manifest keeps the dependency footprint minimal with a single production dependency, `express`.

While the current implementation uses in-memory storage and has not yet wired the Home page to render fetched blog data, these are well-understood and addressable gaps rather than fundamental design flaws. The architecture is deliberately positioned as a strong foundation: adding a database layer, completing the read-and-render flow on the frontend, introducing authentication, and deploying the application to a cloud host would each be incremental steps rather than a rewrite. As such, this project serves its purpose well, both as a learning exercise in full-stack development and as a base that can be iteratively evolved into a production-capable blogging platform.

13. References

- Repository source: <https://github.com/koyyadakrithika-spec/website>
- Express.js official documentation: <https://expressjs.com/>
- Node.js official documentation: <https://nodejs.org/en/docs>
- MDN Web Docs — Fetch API: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
- MDN Web Docs — Using the JSON API: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/JSON
- npm registry — express package: <https://www.npmjs.com/package/express>